

From Trust to Zero Trust: Adaptive Detection of On-Off Attacks in Distributed Fog Systems

Sita Rasagna, G.Geetha kumari

June 2025

1 Introduction

1.1 Performance Trust

Performance trust evaluates how reliably a node completes its assigned tasks. It is computed as:

$$T_{\text{perf}} = \frac{\text{Successful Tasks}}{\text{Total Tasks}} \quad (1)$$

This metric provides direct evidence of node reliability based on its task execution history.

1.2 Feedback Trust

Feedback trust reflects the reputation of a node based on peer evaluations. Let R_{ij} denote the feedback score given by node i about node j , and w_i be the trust weight of node i . Then feedback trust for node j is computed as:

$$T_{\text{fb}}(j) = \frac{\sum_{i \in V} w_i \cdot R_{ij}}{\sum_{i \in V} w_i} \quad (2)$$

To ensure robustness against collusion or ballot stuffing, the feedback data is pre-processed to remove outliers:

Outlier Detection using IQR:

- Compute first ($Q1$) and third quartiles ($Q3$)
- Calculate $IQR = Q3 - Q1$
- Discard feedback outside $[Q1 - 1.5 \cdot IQR, Q3 + 1.5 \cdot IQR]$

Alternatively, lightweight machine learning models such as Isolation Forests can be used to eliminate anomalous ratings.

1.3 Behavioral Trust

Behavioral trust evaluates how efficiently a node behaves during task execution. It penalizes delays and excessive resource consumption.

$$T_{\text{beh}} = 1 - (\text{Normalized Delay} + \text{Overuse Penalty}) \quad (3)$$

Where:

- **Normalized Delay** quantifies the relative delay of a node in completing its task compared to the expected time, scaled to $[0,1]$.
- **Overuse Penalty** captures the excess consumption of CPU, memory, or bandwidth beyond assigned quotas.

A node with efficient and timely execution will have a high behavioral trust score, whereas one that is slow or wasteful will receive a lower score.

1.4 Fused Trust Score

To derive a single unified trust score from multiple components, we define a fused trust metric as follows:

$$T_{\text{fused}} = \sqrt{\frac{T_{\text{perf}} \cdot T_{\text{fb}} \cdot T_{\text{beh}}}{\frac{T_{\text{perf}} + T_{\text{fb}} + T_{\text{beh}}}{3}}} \quad (4)$$

This formula:

- Penalizes imbalance among the components
- Rewards nodes that perform consistently across all trust dimensions
- Limits over-reliance on any one metric

By using both geometric and arithmetic aggregation with a square-root smoothing, it ensures robustness against manipulation and emphasizes balanced node behavior.

1.5 Sliding Window Mechanism

To capture dynamic behavior over time, we maintain a sliding window W of recent trust scores for each node:

$$W = [T_{t-k+1}, T_{t-k+2}, \dots, T_t] \quad (5)$$

Here:

- k is the window size (typically 5 or 10)
- T_t is the fused trust score at current time t

This window helps monitor the trust history of a node and is used for computing temporal anomaly metrics such as oscillation, variance, and change points.

By using a moving window, the system becomes more sensitive to fluctuations in behavior—crucial for detecting on-off attacks and unstable trust patterns.

1.6 Temporal Anomaly Detection

To identify irregular or suspicious trust behavior over time, we define three anomaly scores based on a sliding window of recent trust values.

1. Oscillation Score

$$Osc = \frac{\text{Number of Trust Flips}}{k - 1} \quad (6)$$

This measures how frequently the trust trend switches between increase and decrease. High oscillation suggests unstable or manipulative behavior.

2. Variance Score

$$Var = \frac{\sigma(T)}{\sigma_{\max}} \quad (7)$$

Where $\sigma(T)$ is the standard deviation of trust values in the sliding window, and $\sigma_{\max}$ is the maximum expected variance for normalization.

3. Change Point Score

$$CP = \frac{\text{Number of Detected Change Points}}{c_{\max}} \quad (8)$$

This score tracks abrupt shifts in trust using time-series change point detection methods. Frequent shifts may signal erratic or on-off attack behavior.

1.7 Temporal Anomaly Detection

To identify irregular or suspicious trust behavior over time, we define three anomaly scores based on a sliding window of recent trust values.

1. Oscillation Score

$$Osc = \frac{\text{Number of Trust Flips}}{k - 1} \quad (9)$$

This measures how frequently the trust trend switches between increase and decrease. High oscillation suggests unstable or manipulative behavior.

2. Variance Score

$$Var = \frac{\sigma(T)}{\sigma_{\max}} \quad (10)$$

Where $\sigma(T)$ is the standard deviation of trust values in the sliding window, and $\sigma_{\max}$ is the maximum expected variance for normalization.

3. Change Point Score

$$CP = \frac{\text{Number of Detected Change Points}}{c_{\max}} \quad (11)$$

This score tracks abrupt shifts in trust using time-series change point detection methods. Frequent shifts may signal erratic or on-off attack behavior.

1.8 Adaptive Anomaly Index Calculation

To summarize temporal instability into a single score, we compute an adaptive anomaly index by weighting oscillation, variance, and change-point scores.

Softmax-Based Weighting

Let $O' = Osc$, $V' = Var$, $C' = CP$. Then:

$$Z = e^{O'} + e^{V'} + e^{C'} \quad (12)$$

$$\alpha = \frac{e^{O'}}{Z}, \quad \beta = \frac{e^{V'}}{Z}, \quad \gamma = \frac{e^{C'}}{Z} \quad (13)$$

Final Anomaly Index

$$AnomalyIndex = \alpha \cdot Osc + \beta \cdot Var + \gamma \cdot CP \quad (14)$$

This adaptive method dynamically emphasizes the most relevant anomaly dimension in each time window, enhancing robustness against complex attack behaviors.

1.9 Final Trust Adjustment using Anomaly Index

Even nodes with high trust scores may exhibit volatile or suspicious behavior. To penalize such instability, we compute the final trust score as:

$$T_{\text{final}} = \frac{T_{\text{fused}}}{1 + AnomalyIndex} \quad (15)$$

Where:

- T_{fused} is the fused trust score from all dimensions.
- $AnomalyIndex$ is the dynamic score indicating instability over time.

This formulation ensures that:

- Nodes with stable behavior retain their trust.
- Nodes with frequent or severe behavioral anomalies are penalized, even if their past performance was acceptable.

1.10 Context-Aware Task Assignment using Rule-Based Policy

The final trust score T_{final} and Anomaly Index provide a primary basis for Zero Trust decision-making. However, fixed thresholding may not adapt well to dynamic fog environments. We introduce a **rule-based context-aware policy** to refine task assignment based on real-time operational factors.

Context Dimensions Considered

The following contextual parameters influence the policy:

- **System Load:** Indicates whether the fog environment is under high resource usage.
- **Task Criticality:** Distinguishes between best-effort and safety-critical tasks.
- **Threat Level:** Represents the current security risk posture (e.g., normal or elevated threat).

Rule-Based Task Assignment Matrix

We define a set of rules that combine trust, anomaly, and contextual factors. Each rule triggers a corresponding action:

T_{final}	Anomaly Index	Load	Criticality	Threat	Action
> 0.8	< 0.3	Any	Any	Normal	Full Assignment
$0.6-0.8$	Any	Low	High	Normal	Test Task + Logging
< 0.5	> 0.5	Any	Any	Any	Quarantine
$0.5-0.7$	< 0.3	High	High	Alert	Partial Assignment
> 0.7	< 0.2	High	Low	Normal	Full Assignment with Monitoring

Benefits of Rule-Based Policy

- **Transparency:** Rules are interpretable by administrators.
- **Customizable:** Easy to extend or modify based on environment needs.
- **Lightweight:** No training or complex computation required.

This method ensures Zero Trust compliance while adapting to the realities of fog-based task execution.